Symptom Analysis and Treatment Recommendation System

Qiyue Yue, Cheng Han Chung, Yu-Tai Lee

University of the Pacific

***Abstract*— Older adults often face difficulty accessing and interpreting online health information, while general-purpose chatbots may produce fluent but unreliable answers. This work presents a safety-focused retrieval-augmented question answering (RAG) system designed to provide accessible, non-diagnostic health guidance for seniors. The system combines a curated SeniorHealthQA dataset, SentenceTransformer based retrieval, and a two-stage Mistral/Ollama generation process that summarizes evidence before producing conservative, safety-aware responses. A similarity threshold filters poor matches to reduce hallucinations. Initial qualitative testing suggests that this approach delivers clearer and more grounded explanations than standalone LLM outputs while maintaining strong safety constraints. The system illustrates the potential of lightweight RAG methods for supporting older adults' everyday health information needs.**

Keywords— retrieval-augmented generation, question answering, elderly health information, sentence embeddings, large language models.

I. INTRODUCTION

Population aging is reshaping healthcare demand worldwide. Older adults, especially over 65 years old, face a higher burden of chronic diseases, multimorbidity, and medication complexity, yet they often have lower digital and health literacy. When seniors or their caregivers search for health answers online, they encounter three persistent problems: information overload and inconsistent quality across sources, generic guidance that overlooks age-specific risks, and presentation styles that are either too technical or too simplified to support safe decisions. These gaps make it hard to quickly find reliable, senior-appropriate answers to practical questions such as medication timing, side-effect triage, or lifestyle adjustments around common conditions.

This project targets that gap by building a domain-focused question–answering (QA) system for health. Our goal is not to replace clinical care but to provide fast answers to common questions about elderly diseases while emphasizing verifiable sourcing. Concretely, we focus on curating and refining a senior-health QA corpus (based on our evolving *SeniorHealthQA* dataset), training models that can retrieve, rank, and select age-appropriate answers and enforcing guardrails that discourage out-of-scope advice and surface "see a clinician" cues when warranted.

Technically, we implement a retrieval augmented generation (RAG) pipeline exposed through a Flask web application. User questions are first encoded with a SentenceTransformer model and compared against pre-computed embeddings of all questions in the SeniorHealthQA corpus. If a similar question is found, the corresponding expert-written answer is summarized and passed—together with the user's query—to a Mistral-based large language model, via the Ollama framework, which generates conversational reply. When no good match exists, the system falls back to a general health answer from the same model, still guided by prompts that encourage cautious language and recommend professional care

for urgent issues. This architecture allows us to combine the factual grounding of a medical dataset with modern LLMs.

In summary, our contributions are threefold: we construct and use a focused SeniorHealthQA corpus tailored to common concerns in older adults, we design and implement an embedding-based retrieval module and RAG chatbot backend in Python, and we deploy the system as an interactive web service. The rest of this report details related work in medical QA and retrieval-augmented systems, describes our implementation and testing workflow, and concludes with a discussion of current limitations and directions for future improvement.

A. PROBLEM STATEMENT

General-purpose QA models struggle in senior-health contexts because relevant answers hinge on age-specific, comorbidity interactions, and functional status. Off-the-shelf systems have some issues for example, hallucinate, cite non-authoritative sources, and miss nuances such as renal-dose adjustments, drug–drug interactions in polypharmacy, or fall-risk trade-offs. They also rarely adapt their explanations to the literacy level of older adults and caregivers.

Therefore, we frame the task as safe answer selection for elderly-disease queries: given a user question, the system should identify and generate an answer tailored to older adults, with appropriate cautions. In operational terms, this means retrieving the most relevant entry from our SeniorHealthQA corpus when possible, using that entry as structured context for an LLM, and prompting the model to avoid definitive diagnoses, to highlight uncertainty, and to direct the user to clinicians when red-flag patterns are mentioned. We combine data, semantic retrieval, and LLM-based generation in a modular pipeline:

> Our system is built on the SeniorHealthQA corpus, a structured collection of medical Q&A pairs focused on common issues faced by older adults. Each entry includes a clearly written question and an accessible, non-technical answer, forming both the system's knowledge base and its retrieval index. For semantic search, the chatbot loads the dataset at startup, encodes all questions into dense vectors using the SentenceTransformer model all-MiniLM-L6-v2, and stores these embeddings in memory. When a user submits a query through the Flask API, the system embeds the incoming question, calculates cosine similarity against the stored vectors, and selects the best match if it exceeds a fixed threshold. If the match is strong enough, we retrieve the corresponding answer from clean_QA.xlsx and send it through a summarization step, where a Mistral model served via Ollama condenses the medical content into a concise form. The final response is generated by another Mistral call that incorporates both the summary and the user's question, guided by prompts that reinforce non-diagnostic language and recommend seeing a clinician when symptoms seem serious or uncontrolled. When no suitable match is found, the system instead uses a simplified prompt that asks Mistral to provide a general, safety-aware health explanation. This entire workflow is accessible through a lightweight Flask service, where the /ask endpoint receives a user's question and returns a JSON-formatted answer for easy integration into web or mobile front ends.

B. CONTRIBUTIONS

In this project, we built a focused SeniorHealthQA dataset that captures common concerns among older adults, such as chronic disease management, medication-related questions, and practical day-to-day adjustments. To support accurate information retrieval, we implemented a semantic matching module using SentenceTransformers and cosine similarity to pair user queries with the most relevant entries in the dataset. These retrieved answers are then incorporated into a retrieval-augmented generation pipeline, where they are summarized and passed to a Mistral model served through Ollama to produce clearer, more grounded responses while reducing unnecessary context noise.

To promote safe usage, we added simple guardrails through prompt design and similarity thresholding, helping the system avoid overly confident diagnostic statements and instead encourage users to consult healthcare professionals when appropriate. Finally, we wrapped the entire pipeline in a lightweight Flask API, making the system easy to integrate with web or mobile front-end applications.

C. SCOPE AND LIMITATIONS

The system is intended for information support, not diagnosis or individualized treatment. It cannot replace clinician judgment, and it may be conservative (preferring "seek care" recommendations) when red-flag patterns are detected. Domain coverage is currently centered on common senior conditions and will expand as the dataset grows.

D. OUTCOME

By aligning data, modeling, and safety design with geriatric needs, the system helps older adults and caregivers ask better questions in the clinic and make more informed decisions between visits.

II. RELATED WORK

Our project lies at the intersection of several practical domains, including general online health information, basic FAQ-style conversational systems, and recent developments involving embeddings and large language models. In this section, we briefly outline the relevant background and clarify how our system both relates to and differs from existing approaches across these areas.

A. ONLINE HEALTH INFORMATION FOR SENIORS

Most older adults and caregivers still depend on general health websites such as hospital pages, public health portals, and search engines to look up information about diseases or medications. While these sources can be helpful, the information is scattered across many pages, often written with technical language, and usually geared toward the average adult rather than seniors who may face multiple health conditions or complex medication needs.

Our project keeps the idea of using written medical resources as the foundation, but we reorganize this information into a single, structured SeniorHealthQA dataset. Instead of requiring users to search and filter through multiple sites, our system provides a chatbot that delivers direct, focused answers tailored to the needs of older adults and their caregivers.

B. FAQ AND RULE-BASED CHATBOTS

A common design seen in tutorials and blog examples is the simple FAQ-style chatbot, where the system stores a collection of question–answer pairs and retrieves the closest match when a user submits a query. Earlier versions rely on keyword matching or TF-IDF, while more recent approaches use sentence embeddings and cosine similarity to achieve better semantic alignment.

Our project largely follows this framework. We compile senior-health question–answer pairs in clean_QA.xlsx, embed both stored questions and user queries using the SentenceTransformer model all-MiniLM-L6-v2, and then compute cosine similarity to identify the best match.

Where our system differs slightly is in how it generates the final response. Instead of returning the original answer text, we use an LLM "Ollama" to summarize and rephrase the content, producing a more conversational reply for older adults.

C. LLM-BASED MEDICAL AND HEALTH CHATBOTS

With the growing use of large language models, many demo projects now show medical Q&A chatbots that simply pass the user's question to an LLM and let it generate an answer. This approach is easy to build, but it also comes with clear risks: the model may produce information that is not grounded in reliable medical sources, and different runs of the same question can lead to inconsistent outputs—both of which are problematic in a health-related setting.

Our project is much smaller in scale, but we still try to reduce these issues rather than relying on the LLM alone. When the system finds a high-similarity match in the SeniorHealthQA dataset, that written answer becomes our primary reference. The LLM Ollama is used only to rewrite the content in a shorter way.

Only when the similarity score drops below our threshold, 0.45 in the code, do we allow the model to answer more freely. Compared with advanced medical LLM systems, we do not perform full evidence retrieval or run additional safety filters.

D. RETRIEVAL-AUGMENTED GENERATION AND SEMANTIC SEARCH

Many GitHub examples and online tutorials describe a basic retrieval-augmented generation (RAG) workflow: documents or FAQs are embedded, the system retrieves the most relevant ones for a given query, and an LLM uses those retrieved texts to produce an answer. Our project follows this same idea, but in a much simpler form.

In our setup, each question–answer pair in the SeniorHealthQA dataset is treated as a small "document." We embed only the questions, use cosine similarity to select the single closest match, and—if the similarity is high enough—send the paired answer into our summarize() and generate_answer() functions, which ultimately call Mistral via Ollama. This logic is served through a lightweight Flask backend, allowing front-end code to send a POST request to /ask and receive a JSON reply.

E. HOW OUR PROJECT FITS IN

In general, earlier work and tutorials point out several useful ideas: older adults often struggle to navigate scattered online health resources, simple FAQ-style chatbots can link user questions to stored answers, large language models can produce smooth explanations but sometimes hallucinate, and RAG pipelines help tie model outputs back to retrieved text. These insights form the background for our project.

Building on them, we created a small SeniorHealthQA table focused on common health questions among older adults and applied sentence embeddings with cosine similarity to retrieve the closest match for each user query. Once a match is found, Mistral (via Ollama) is used mainly to summarize or rephrase the existing answer rather than generate new medical content.

The full workflow runs through a lightweight Flask API, which makes it easy to test in a browser or integrate with a simple front-end. Overall, the design stays manageable for a university project while still incorporating modern ideas such as semantic retrieval and retrieval-augmented generation in a controlled, small-scale setting.

III. IMPLEMENTATION DETAILS

A. DATA: SENIORHEALTHQA CORPUS

Our system is built around a small question–answer corpus focused on elderly health. Before coding, we manually cleaned the raw medical text by removing duplicates, fixing basic formatting issues, and ensuring each question aligned with one clear, non-technical answer. The final dataset, stored in clean_QA.xlsx, contains 591 QA pairs across two columns ("question" and "answer"). This file serves both as the knowledge base and the retrieval index, since all questions are later embedded for similarity search.

B. TOOLS, FRAMEWORKS, AND PROGRAMMING LANGUAGES

The system is implemented in Python, with most experiments first tested in Jupyter notebooks. We use pandas for loading the Excel file, SentenceTransformers (all-MiniLM-L6-v2) to embed questions, and Ollama to run the Mistral language model locally. A lightweight Flask server exposes the system through two routes: a simple index page and a /ask endpoint that returns JSON answers. Overall, the stack is intentionally minimal—one dataset, one embedding model, one LLM, and a small API.

C. SYSTEM ARCHITECTURE AND WORKFLOW

At startup, all questions in SeniorHealthQA are embedded once and stored in memory. When a user submits a query, we embed the question, compute cosine similarity with all stored embeddings, and select the best match. If the similarity score is above 0.45, we treat the corresponding answer as the reference text: it is briefly summarized and then rephrased by Mistral to produce a clear, conversational reply. If no strong match is found, the model switches to a fallback prompt asking Mistral to answer the question cautiously without giving diagnostic advice.

D. TESTING ENVIRONMENT AND WORKFLOW

Testing was done on a standard student laptop with Python, Ollama, and Flask installed. We manually checked the dataset, verified embedding behavior in notebooks, and tested both in-domain and out-of-domain queries. End-to-end testing involved running the Flask server locally and interacting with the interface through a browser. Since this is a course project, evaluation was mostly qualitative—focusing on whether responses were coherent, concise, and reasonably safe.

E. USER INTERFACE

To demonstrate the system's practical usability, we implemented a lightweight web-based interface that allows users—particularly older adults—to interact with the RAG-powered health information assistant. The design(fig1) uses large fonts, warm low-contrast colors, and a simplified chat layout to support older adults who may have limited vision or digital literacy. The initial system message reinforces the non-diagnostic nature of the tool and encourages users to seek professional care when appropriate.

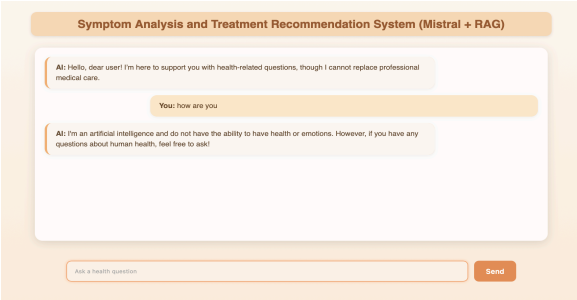

Fig 1. User interface

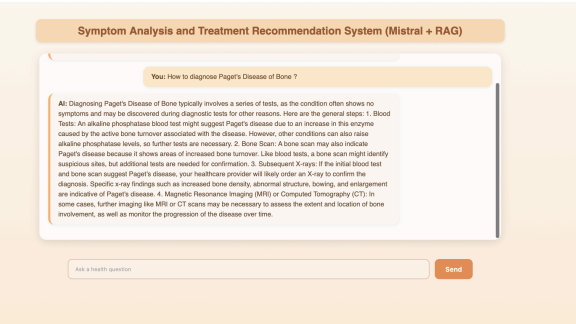

Fig 2. AI chatbot responds

Figure 2 provides an example of a more complex query involving a diagnostic topic. The system retrieves structured information and presents it as step-by-step explanations while still avoiding direct diagnosis.

F. SAMPLE INPUTS AND OUTPUTS

When the user asks a question that closely matches an entry in SeniorHealthQA (e.g., "How many people are affected by osteoarthritis?"), the system retrieves the aligned answer, summarizes it, and returns a friendly explanation. For questions not covered in the dataset (e.g., medication safety or lifestyle concerns), the system falls back to a general health prompt and asks Mistral to generate a careful, non-diagnostic response.

IV. CONCLUSION

In this project, we built a small, domain-focused question–answering system for elderly health based on a SeniorHealthQA dataset, sentence embeddings, and a locally served Mistral language model. Our goal was not to create a production medical assistant, but to explore how modern NLP tools can be combined in a controlled way to provide clearer and more focused answers for older adults and their caregivers. Technically, we showed that a simple semantic retrieval pipeline using the all-MiniLM-L6-v2 SentenceTransformer and cosine similarity is enough to match many user questions to appropriate entries in a manually cleaned QA table. On top of this, using Mistral via Ollama to

summarize and rephrase retrieved answers produces shorter, more conversational explanations, and wrapping the logic in a lightweight Flask API makes the system easy to test through a web interface.

At the same time, the system has important limitations. The SeniorHealthQA dataset is relatively small and covers only a subset of diseases and topics, so many real-world questions will not have a close match and will fall back to a more generic LLM answer. The system has no access to personal medical data, so all responses are educational and cannot be treated as individualized clinical advice. Safety mechanisms are basic: we rely mainly on cautious prompting and conservative wording, without a dedicated triage or emergency detector. Finally, our evaluation is qualitative and informal, based on manual testing rather than a structured user study or quantitative metrics, and the interface is a simple English-only web page rather than a fully optimized, senior-friendly UI.

These limitations suggest clear directions for future work. The SeniorHealthQA corpus could be expanded and refined, especially for common geriatric syndromes and frequently asked medication questions, to improve retrieval quality. Safety could be strengthened by adding explicit rules or lightweight classifiers to detect red-flag symptoms and respond with clear "seek immediate care" messages. More systematic evaluation, potentially involving medical students, clinicians, or older adults, would help measure usefulness, clarity, and perceived safety. Finally, the interface could be redesigned with accessibility in mind (larger fonts, simpler layout, voice options, bilingual support), turning this student prototype into a more realistic tool for supporting health literacy and shared decision-making in an aging population.

REFERENCES

[1] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," arXiv preprint arXiv:1908.10084, 2019. Available: https://arxiv.org/abs/1908.10084

[2] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020. Available: https://papers.nips.cc/paper_files/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[3] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020. Available: https://arxiv.org/abs/2005.11401

[4] World Health Organization, "Health literacy," Fact Sheet, Aug. 2024. Available: https://www.who.int/news-room/fact-sheets/detail/health-literacy

[5] Flask Project, "Flask Documentation (3.1.x)," Available: https://flask.palletsprojects.com/

[6] Pandas Development Team, "pandas: Python Data Analysis Library," Available: https://pandas.pydata.org/

[7] SentenceTransformers Team, "SentenceTransformers Documentation," Available: https://www.sbert.net/

[8] SentenceTransformers, "all-MiniLM-L6-v2: Sentence-Transformers Model Card," Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[9] Mistral AI, "Mistral 7B Model Card (Mistral-7B-v0.1)," Available: https://huggingface.co/mistralai/Mistral-7B-v0.1

[10] Ollama, "Ollama Documentation," Available: https://docs.ollama.com/

[11] Ollama, "Ollama: Chat & Build with Open Models," Available: https://ollama.com/